

Представление объектного файла: C++, Rust, Zig

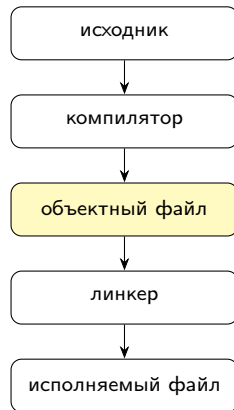
Форматы, символы, релокации, ABI и отладочная информация

Аржаев Дмитрий

29.09.2026

Зачем смотреть на объектный файл

- Объектный файл («.o») — результат компиляции одной единицы трансляции; вход линкера [2, гл. 1–3].
- Именно в нём фиксируется всё, что язык обещает внешнему миру: имена, ABI, режимы связывания.
- Три языка — C++, Rust, Zig: разные модели связывания и разная степень стандартизации ABI.



План

- 1 Обзор: форматы, секции, эволюция стандартов.
- 2 Особенности C++/Rust/Zig на собственных примерах.
- 3 Методология: метрики и станд.
- 4 Результаты сравнения.

Вопросы

- Как различаются размер и состав .o?
- Сколько работы остаётся линкеру?
- Насколько совместим каждый язык с C ABI?
- Воспроизводим ли результат?

Форматы объектных файлов: эволюция и сравнение

- Линия развития: `a.out` → COFF → ELF; PE выросла из COFF; Mach-O — формат семейства NeXTSTEP/macOS/iOS [2, 5, 6].
- Объектный файл на разных ОС: ELF (ET_REL) [1], COFF (.obj) [5], Mach-O (MH_OBJECT) [6].

	ELF [1]	PE/COFF [5]	Mach-O [6]
ОС	Linux, BSD, Solaris	Windows	macOS, iOS
Единица	секции	секции	сегменты → секции
Имя кода	.text	.text	__TEXT, __text
Каталог	таблица секций	таблица секций	load commands

Структура relocatable ELF (.o)

ELF header
.text
.data / .bss / .rodata
.symtab / .strtab
.rela.*
.debug_* / .eh_frame
таблица заголовков секций

- ELF header: класс (32/64), порядок байт, машина, тип файла [1].
- Таблица секций описывает содержимое; для .o она главная [1].
- Таблица сегментов (program headers) нужна загрузчику; у relocatable-файла обычно отсутствует [1].
- .bss (NOBITS) не занимает места в файле.
- Подробнее: `man 5 elf`.

Секция [1]	Назначение
<code>.text</code>	машинный код
<code>.data, .bss</code>	инициализированные / нулевые изменяемые данные
<code>.rodata</code>	константы, строковые литералы
<code>.symtab, .strtab</code>	таблица символов и строк
<code>.rela.text, .rela.*</code>	записи релокаций
<code>.eh_frame, .gcc_except_table</code>	раскрутка стека и обработка исключений [4]
<code>.group / COMDAT</code>	дедупликация inline-функций и шаблонов [4]
<code>.debug_*</code>	отладочная информация DWARF [7]
<code>.note.GNU-stack</code>	признак исполняемости стека [13]

Что стандарты говорят про объектный файл

- **C++**: стандарт ISO/IEC 14882 описывает фазы трансляции, linkage и ODR, но не формат .o. Формат и ABI задают Itanium C++ ABI, System V psABI и MSVC [8, 4, 3].
- **Rust**: ISO-стандарта нет; поведение описано в Rust Reference и RFC (например, RFC 2603 про mangling) [15, 14].
- **Zig**: язык описан в Language Reference; до версии 1.0 стабильность ABI и формата не гарантируется [17].

Вывод: единственный «общий язык» между тремя компиляторами — C ABI платформы.

Как развитие C++ влияло на объектный файл

Стандарт	Что появилось	Следствие для .o
C++98	ODR, extern "C", шаблоны, inline [9]	mangling, weak/COMDAT-символы [4]
C++11	thread_local	TLS-секции и TLS-релокации [11]
C++17	inline-переменные [9]	COMDAT-группы для данных [4]
C++20	модули [9]	BMI как новый артефакт рядом с .o; формат не стандартизован [9]

- Запись символа: имя, значение, размер, тип, binding, visibility, индекс секции [1].
- Binding: LOCAL, GLOBAL, WEAK. Тип: FUNC, OBJECT, TLS, SECTION, FILE [1].
- Индекс UND — символ не определён в этом файле; его разрешит линкер [1].

Просмотр: `readelf -sW file.o, nm -C file.o, llvm-readobj --symbols file.o`.

Свой пример: C++

```
1 namespace geom {  
2     int area(int w, int h) {  
3         return w * h;  
4     }  
5 }  
6 template <class T>  
7 T add(T a, T b) {  
8     return a + b;  
9 }  
10 template int add<int>(int, int);  
11 extern "C" int c_area(int w, int h) {  
12     return geom::area(w, h);  
13 }
```

Смотрим символы: `geom::area`, `add<int>`, `c_area`.

Свой пример: Rust 2024

```
1 pub mod geom {
2     pub fn area(w: i32, h: i32) -> i32 {
3         w * h
4     }
5 }
6 pub fn add<T: core::ops::Add<Output = T>>(a: T, b: T) -> T {
7     a + b
8 }
9 pub fn add_i32(a: i32, b: i32) -> i32 {
10     add(a, b)
11 }
12 #[unsafe(no_mangle)]
13 pub extern "C" fn c_area(w: i32, h: i32) -> i32 {
14     geom::area(w, h)
15 }
```

Смотрим символы: `geom::area`, `add_i32` (в него подставлен `add::<i32>`), `c_area`.

Свой пример: Zig

```
1 const geom = struct {  
2     fn area(w: i32, h: i32) i32 {  
3         return w * h;  
4     }  
5 };  
6 fn add(comptime T: type, a: T, b: T) T {  
7     return a + b;  
8 }  
9 export fn add_i32(a: i32, b: i32) i32 {  
10     return add(i32, a, b);  
11 }  
12 export fn c_area(w: i32, h: i32) i32 {  
13     return geom.area(w, h);  
14 }
```

Экспортируются только `add_i32` и `c_area`; `geom.area` и `add` остаются внутренними.

Как смотреть символы и релокации

Символы

```
readelf -sW file.o  
nm -C file.o  
llvm-readobj --symbols file.o
```

Релокации

```
readelf -rW file.o  
objdump -dr file.o
```

Инструменты: GNU binutils [20] и LLVM [21].

Mangling: C++ (Itanium ABI)

- Имя кодирует область видимости, имя и типы параметров: префикс `_Z`, вложенные имена в `N...E` [4].
- `extern "C"` отключает mangling: остаётся имя `c_area` [4].

Сущность	Символ
<code>geom::area(int, int)</code>	<code>_ZN4geom4areaEii</code>
<code>add<int>(int, int)</code>	<code>_Z3addIiET_S0_S0_</code>
<code>c_area (extern "C")</code>	<code>c_area</code>
<code>vtable / typeid</code> класса	префиксы <code>_ZTV</code> , <code>_ZTI</code>

Проверка: `c++filt, nm` на `.o`, Compiler Explorer [19].

- **Rust v0** (RFC 2603): префикс `_R` и структурированное кодирование пути и параметров символа [14]. Стабильный `rustc` использует эту схему по умолчанию начиная с Rust 1.97; версию `rustc` нужно зафиксировать в эксперименте [16].
- **Rust legacy**: историческая схема с именами вида `_ZN4demo4geom4area17h<16 hex>E`; до Rust 1.97 была схемой по умолчанию [16].
- **Rust FFI**: `#[unsafe(no_mangle)]` и `extern "C"` — имя экспортируемого символа задаётся явно [15].
- **Zig**: `export fn` даёт символ с исходным именем (`c_area`); неэкспортируемые функции остаются внутренними [17].

- Компилятор не знает финальных адресов, поэтому оставляет в `.rela.*` записи: смещение, тип, символ, addend [1].
- Линкер вычисляет значение по формуле типа [3]: S — адрес символа, A — addend, P — изменяемое место, L — запись в PLT, G — запись в GOT, GOT — адрес GOT.

Тип (x86-64)	Формула	Где встречается
R_X86_64_64	$S + A$	64-битный абсолютный адрес [3]
R_X86_64_PC32	$S + A - P$	доступ к данным относительно PC [12]
R_X86_64_PLT32	$L + A - P$	вызов внешней функции [12]
R_X86_64_GOTPCREL	$G + GOT + A - P$	доступ через GOT [12]
R_X86_64_TPOFF32	TLS-смещение	thread-local в исполняемом файле [11]

Все три языка собираются через LLVM-бэкенд, поэтому наборы релокаций близки, но не обязаны совпадать. Их число зависит от PIC/PIE, TLS и отладочной информации (в `.rela.debug.*` их много) [13].

	System V AMD64 [3]	Microsoft x64 [5]
Целочисленные аргументы	rdi, rsi, rdx, rcx, r8, r9	rcx, rdx, r8, r9
Вещественные аргументы	xmm0–xmm7	xmm0–xmm3
Возврат	rax (rdx), xmm0	rax, xmm0
Shadow space	нет	32 байта

- **C++**: соглашение платформы; на Linux с GCC/Clang layout классов и vtable задаёт Itanium C++ ABI, на Windows используется ABI MSVC [4].
- **Rust**: `extern "Rust"` — ABI не является стабильным контрактом. Для FFI используют `extern "C"` (ABI функций) и `#[repr(C)]` (layout типов) [15].
- **Zig**: соглашение задаётся через `callconv` (`.C` в старых версиях, `.c` в новых); `export` использует C ABI [17].

Исключения, RTTI и метаданные

	C++ [4]	Rust [15]	Zig [17]
Ошибки	исключения	Result; panic (unwind или abort)	error union (значение)
Раскрутка стека	.eh_frame, LSDA	.eh_frame при panic=unwind	не требуется для ошибок
RTTI	typeinfo, vtable-символы	нет аналога; TypeId на уровне языка	нет
Метаданные модулей	BMI (C++20), вне .o	метаданные крейта (rlib), вне .o	нет

Практическая проверка: сравнить `.eh_frame`, `.gcc_except_table` и число символов `_ZTV/_ZTI` на одном коде с `-fno-exceptions / panic=abort`.

- ELF: DWARF (v5 в современных компиляторах): `.debug_info`, `.debug_abbrev`, `.debug_line`, `.debug_str`, `.debug_line_str`, `.debug_rnglists` [7].
- Windows: CodeView/PDB. macOS: DWARF в объектных файлах, сборка в dSYM.
- Управление: `-g`, `-gsplit-dwarf`, сжатие секций; Rust: `-C debuginfo=0|1|2` [16]; Zig: режим `Debug` содержит отладочную информацию [17].
- В `.o` DWARF сильно нагружает линкер: релокации ссылаются на `.text` и строки.

Метрика	Инструмент	Пояснение
Размер .о и по секциям	<code>size -A, readelf -SW</code>	total, .text, .data, .rodata, .debug_*, .eh_frame
Число секций	<code>readelf -SW</code>	—
Символы	<code>nm, readelf -sW</code>	defined / undefined; local / global / weak
Релокации	<code>readelf -rW</code>	число и типы; с -g и без

Сложность ABI (mangling, исключения, RTTI, метаданные) не сводится к одному числу и представлена качественной таблицей. Инструменты: binutils [20], LLVM [21], заметки [13].

Метрика	Инструмент	Пояснение
Время и память компиляции	hyperfine [22], /usr/bin/time -v	медиана и разброс, Max RSS
Работа линкера	размер после --gc-sections, LTO [10]	undefined, релокации, DCE
Совместимость с C ABI	чек-лист	вызов из C, вызов C, стабильность
Воспроизводимость	sha256sum	две сборки в разных каталогах

Метрики фиксируют как свойства объектного файла, так и стоимость сборки и последующей линковки.

- Платформа: x86-64 Linux, ELF; clang++ 22.1.8, rustc 1.98.1, zig 0.16.0; readelf 2.47.
- В сырых данных присутствуют режимы 00g, 02, 02g, 0s; таблицы report.py по умолчанию строятся только для 02 и 02g, а графики — для 02g.
 - C++: -00, -02, -0s;
 - Rust: -C opt-level=0|2|s [16];
 - Zig: Debug, ReleaseFast, ReleaseSmall [17].
- Корпус: 5 программ на каждый язык: demo, generics, strings, errors, cabi.

```
# C++
clang++ -c -O2 -g src.cpp -o out.o

# Rust
rustc --crate-type=lib --emit=obj -C opt-level=2 \
  -C debuginfo=2 -C codegen-units=1 src.rs -o out.o

# Zig
zig build-obj src.zig -O ReleaseFast -fllvm \
  -femit-bin=out.o
```

Для честности сравнения в стенде используется LLVM-бэкенд у всех трёх (в Zig явно `-fllvm`) [17]; у Rust задано `codegen-units=1`, чтобы получить один `.o` [16]. Это ограничение эксперимента, а не свойство языков.

- Собираем один и тот же исходник дважды: в разных каталогах и в разное время [18].
- Сравниваем sha256 объектных файлов; при отличии смотрим diff вывода `readelf/objdump`.
- Типичные источники различий: абсолютные пути в DWARF, метки времени, хэши в символах [18].
- Инструменты: `-ffile-prefix-map` (C++) [18], `--remap-path-prefix` (Rust) [16].

Размер объектного файла

Язык / режим	total	.text	data+rodata	.debug_*	.eh_frame
C++ -O2	?	?	?	?	?
Rust -O2	?	?	?	?	?
Zig ReleaseFast	?	?	?	?	?

График появится, когда будет создан `results/sizes.csv`
(колонки: lang,text,data,rodata,debug).

Язык / режим	секций	символов	weak	релокаций	основной тип
C++ -O2	?	?	?	?	?
Rust -O2	?	?	?	?	?
Zig ReleaseFast	?	?	?	?	?

Время и память компиляции; работа линкера

Язык	время, мс (медиана)	Max RSS, МБ	размер после --gc-sections	с LTO
C++	?	?	?	?
Rust	?	?	?	?
Zig	?	?	?	?

Сравнение по ABI и совместимости (качественно)

	C++ [4]	Rust [15]	Zig [17]
Mangling	Itanium ABI (Linux/Clang)	v0 (по умолчанию с Rust 1.97)	без mangling для export
Стабильный ABI языка	нет (ABI задают платформа и реализация)	нет; для FFI используют C ABI	нет
Вызов из C без обёрток	extern "C"	extern "C" + no_mangle	export
Исключения / раскрутка	есть	panic=unwind	нет для ошибок
RTTI-символы	есть	нет	нет
Метаданные вне .o	BMI (модули)	rlib-метаданные	нет

Таблицу нужно проверить на выбранных версиях компиляторов.

Выводы

- [TODO: Вывод 1: что показали размеры и секции (после измерений)]
- [TODO: Вывод 2: символы и релокации]
- [TODO: Вывод 3: цена компиляции]
- [TODO: Вывод 4: воспроизводимость]

Гипотезы для проверки (не результаты)

Шаблоны C++ дают больше weak/COMDAT-символов; в символах Rust путь и параметры инстанцииции закодированы, а одноимённые сущности различаются хешем (legacy) или дизамбигуатором (v0); Zig даёт наименьшее число служебных секций.

Ограничения

Одна архитектура и ОС, LLVM-бэкенд, небольшой корпус, конкретные версии компиляторов.

- Объектный файл — точка, где встречаются язык, компилятор, ABI и линкер.
- Общий знаменатель трёх языков — C ABI; всё остальное определяется реализацией.
- Репозиторий: исходники, скрипты сбора метрик, сырые данные доступны в **репозитории**.

- [1] TIS Committee. *Tool Interface Standard (TIS) Executable and Linking Format (ELF) Specification*, v1.2, 1995.
- [2] J. R. Levine. *Linkers and Loaders*. Morgan Kaufmann, 2000.
- [3] System V Application Binary Interface, AMD64 Architecture Processor Supplement (x86-64 psABI). gitlab.com/x86-psABIs.
- [4] Itanium C++ ABI. itanium-cxx-abi.github.io/cxx-abi.
- [5] Microsoft. PE Format; x64 calling convention. learn.microsoft.com.
- [6] Apple. *Mach-O File Format Reference* (архивная документация).
- [7] DWARF Debugging Information Format, Version 5. dwarfstd.org.
- [8] ISO/IEC 14882 — Programming languages — C++.

- [9] cppreference.com: inline specifier, linkage, modules.
- [10] I. L. Taylor. Linkers (серия статей). airs.com/ian, 2007–2008.
- [11] U. Drepper. *ELF Handling For Thread-Local Storage*, 2013.
- [12] E. Bendersky. Position Independent Code (PIC) in shared libraries on x64. eli.thegreenplace.net.
- [13] F. Song. MaskRay blog. maskray.me.
- [14] Rust RFC 2603: Symbol Name Mangling v0.
- [15] The Rust Reference (type layout, ABI, linkage).
- [16] The rustc book: codegen options, symbol mangling.
- [17] Zig Language Reference. ziglang.org/documentation.

- [18] Reproducible Builds: documentation. reproducible-builds.org.
- [19] Compiler Explorer. godbolt.org.
- [20] GNU Binutils documentation. sourceware.org/binutils/docs.
- [21] LLVM Command Guide (llvm-readobj и др.). llvm.org/docs/CommandGuide.
- [22] D. Peter. hyperfine: command-line benchmarking tool. github.com/sharkdp/hyperfine.